

Experiment 02 · USART - Synchronous (Clocked) Serial Link

USART1 driving a data line and a clock line, captured on the slave by SPI2

NUCLEO-F446RE
× 2
STM32F446RET6,
Cortex-M4

USART1 sync +
SPI2 slave
peripheral used

9.6 kHz bit clock
line settings

3 (TX, CK, GND)
wires between boards

Bare metal
CMSIS registers, no
HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards talk to each other over **USART** and nothing else. The board holding the **master** listing reads its blue B1 user button and measures how long it is held; the board holding the **slave** listing has no button and reacts only to what arrives on the USART link, blinking its green LD2 in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Byte transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	'S' (0x53)	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	'L' (0x4C)	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are the two literals in the slave's `blink()` call, so the timing can be retuned by editing one line. The master flashes its own LD2 for 100 ms each time it sends, so you can always see which board acted.

1.2 How short these programs are

Listing	Lines of code	What it does
Master	about 30	clock enables, four GPIO lines, peripheral init, one loop
Slave	about 25	clock enables, two GPIO lines, peripheral init, one loop

There is no interrupt handler, no clock-tree configuration, no acknowledgement protocol and no checksum. Everything that remains is something an examiner can point at and ask you to justify – which is exactly what you want when you are typing from memory.

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one typed with the master listing, one with the slave listing
1 set	Male-male jumper wires (Dupont)	keep them under about 20 cm
2	USB Mini-B cables	for programming, then for standalone power
1–2	USB power bank or 5 V phone charger	to run the boards away from the PC
2	4.7 kΩ resistors + small breadboard	I2C experiment only – mandatory bus pull-ups

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PA9	USART1_TX	AF7	CN10-21	D8	master: data out
PA8	USART1_CK	AF7	CN10-23	D7	master: bit clock out
PB15	SPI2_MOSI	AF5	CN10-26	-	slave: data in
PB13	SPI2_SCK	AF5	CN10-30	-	slave: clock in
PC13	B1 user button	input + pull-up	CN7-23	-	master only
PA5	LD2 user LED	output	CN10-11	D13	both boards
GND	0 V reference	-	CN10-20	-	mandatory

Reading the headers

CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board; odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, find a pin by its *name*, not by counting rows.

4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the table are the ones used in Figure 1 on the next page; any colour works electrically, but keeping a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Wire colour used	Topology
Data	PA9 USART1_TX · CN10-21	PB15 SPI2_MOSI · CN10-26	red	one direction
Clock	PA8 USART1_CK · CN10-23	PB13 SPI2_SCK · CN10-30	orange	one direction
Ground	GND · CN10-20	GND · CN10-20	black	common

Experiment 2 · USART synchronous - board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.

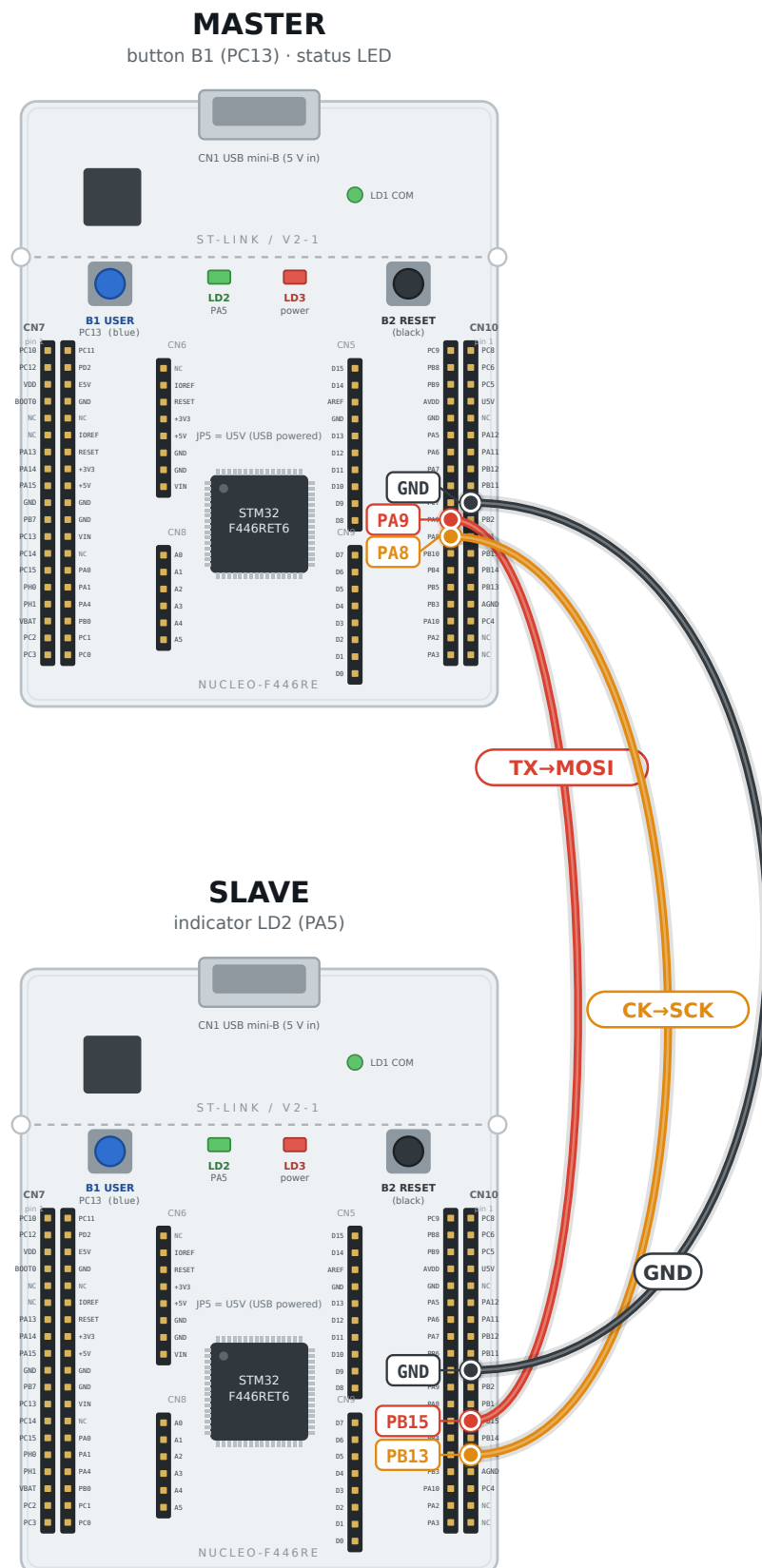


Figure 1 - Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

Setting one bit, `CR2.CLKEN`, turns the USART into a **synchronous** transmitter: it now drives a bit clock on `USART_CK` that is rigidly aligned with the data on `TX`. Three neighbouring bits shape that clock, and all four live in `CR2`:

- **CPOL = 0** (bit 10, left at zero) – the clock idles low between characters.
- **CPHA = 1** (bit 9) – data is captured on the second clock transition of each bit, i.e. in the middle of a stable bit.
- **LBCL = 1** (bit 8) – a clock pulse is produced for the last (8th) data bit too; without it the eighth bit would arrive unclocked.

These four bits must be written while the transmitter is still disabled, which is why the listing writes `CR2` before `CR1`. Get that order wrong and the clock pin stays silent.

The fact that decides this experiment's design

On every STM32, `USART_CK` is an **output only**. A USART can be the synchronous *master* but never the synchronous *slave* – its receiver cannot be driven by an external clock. So the second board cannot receive this stream with its own USART.

The slave therefore captures it with **SPI2 in slave mode**, which is precisely a shift register clocked from outside. Three settings make SPI byte-compatible with a USART synchronous character: `CPHA = 1` and `CPOL = 0` to match the clock shape, and `LSBFIRST = 1` because a USART always sends the least-significant bit first while SPI defaults to most-significant first. Omit `LSBFIRST` and every byte arrives bit-reversed – 'L' (0x4C) would read as 0x32.

Why no NSS wire is needed

The slave sets `SSM = 1` with `SSI = 0`, which tells the peripheral to ignore the NSS pin and behave as permanently selected. `USART_CK` pulses only during the eight data bits – the start and stop bits carry no clock – so the shift register receives exactly eight clocks per character and delivers one clean byte with no chip-select framing at all.

Why there is no clock-configuration code

After reset the STM32F446 runs from its internal 16 MHz RC oscillator (HSI) with every bus prescaler at /1, and CubeIDE's `SystemInit()` does not change that – it only enables the FPU. So **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** without a single line of setup. That single fact is where the two constants in the listing come from: 16000 in `delay_ms()` (16 MHz / 1 kHz) and the peripheral timing value in the init.

Do not paste a `SystemClock_Config()` from a HAL example into these projects. If you raise SYSCLK, the delays and the baud rate move with it.

5.2 The delay function, without an interrupt

All eight listings share the same six-line timer. SysTick is a 24-bit down-counter inside the Cortex-M4 core, so it needs no RCC clock enable and no NVIC setup:

```
SysTick->LOAD = 16000 - 1;    /* reload = 1 ms at 16 MHz */
SysTick->VAL = 0;             /* clear the counter */
SysTick->CTRL = 5;            /* bit0 ENABLE, bit2 CLKSOURCE = core clock */
while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16))); /* COUNTFLAG */
SysTick->CTRL = 0;
```

COUNTFLAG (bit 16) is set every time the counter reaches zero and is **cleared by reading CTRL**, so the inner loop consumes exactly one millisecond per pass. Because there is no interrupt there is no `SysTick_Handler`, no `volatile` global and no chance of forgetting the handler name – three fewer things to get wrong under exam pressure.

The press length is then measured by counting those milliseconds while the button is held: `while (!(GPIOC->IDR & (1<<13))) { delay_ms(10); t += 10; }`, giving a resolution of 10 ms, which is far finer than the 1000 ms decision threshold.

5.4 Register configuration summary

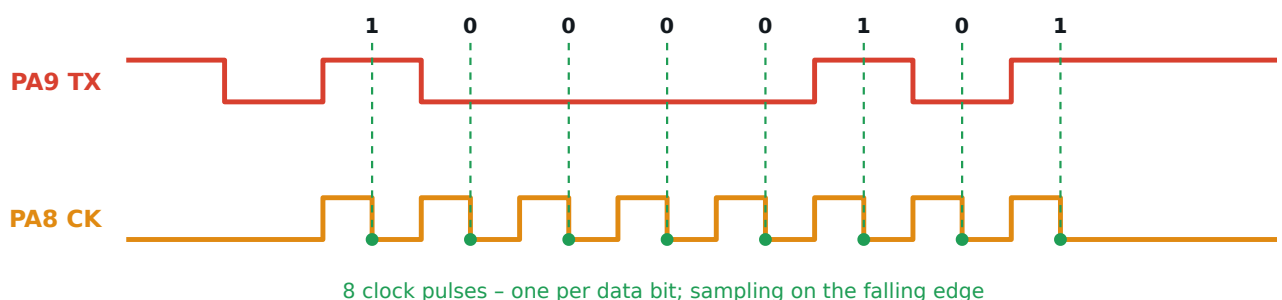
Register	Value written	Why
RCC->APB2ENR	(1<<4)	USART1 clock (master side)
RCC->APB1ENR	(1<<14)	SPI2 clock (slave side) - note APB1, not APB2
GPIOA->AFR[1]	(7<<0) (7<<4)	AF7 puts USART1_CK on PA8 and USART1_TX on PA9
USART1->CR2	(1<<11) (1<<9) (1<<8)	CLKEN, CPHA, LBCL - written before TE is set
USART1->BRR	1667	9600 bit/s, so CK runs at 9.6 kHz
USART1->CR1	(1<<3) (1<<13)	TE and UE - transmitter only, nothing is received on this link
GPIOB->AFR[1]	(5<<20) (5<<28)	AF5 puts SPI2_SCK on PB13 and SPI2_MOSI on PB15
SPI2->CR1 (slave)	(1<<0) (1<<7) (1<<9) (1<<6)	CPHA, LSBFIRST, SSM (with SSI = 0 → selected), SPE; MSTR stays 0
SPI2->SR	bit 0 RXNE	one captured byte is waiting in DR

5.3 Minimum register sequence - memorise this order

MASTER	SLAVE
1. RCC->AHB1ENR : GPIOA, GPIOC 2. RCC->APB2ENR : USART1 3. PC13 pull-up, PA5 output 4. PA8 and PA9 → alternate function, AFRH = 7 5. CR2 = CLKEN CPHA LBCL first 6. BRR = 1667, then CR1 = TE UE 7. loop: read PC13, time it, wait TXE, write DR	1. RCC->AHB1ENR : GPIOA, GPIOB 2. RCC->APB1ENR : SPI2 3. PA5 output 4. PB13 and PB15 → alternate function, AFRH = 5 5. CR1 = CPHA LSBFIRST SSM SPE 6. (no BRR - a slave has no baud-rate field) 7. loop: wait RXNE, read DR, blink

6 Signal timing

USART1 synchronous mode - CPOL = 0, CPHA = 1, LBCL = 1



The clock idles low, rises at the start of each data bit and falls in its middle, where the receiver samples.
START and STOP carry no clock pulse, so a shift-register receiver sees exactly 8 bits per character.

Figure 2 – Synchronous mode. PA8 now carries a bit clock generated by the transmitter itself, so the receiver never has to guess where a bit begins.

7 MASTER source code `usart_sync_master.c`

Type this over the whole contents of `Core/Src/main.c` in the master project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`usart_sync_master.c`

```
1/* EXP 2  USART SYNCHRONOUS (USART1 + CK)  --- MASTER ---  NUCLEO-F446RE
2 * Wiring : PA9 TX (CN10-21) -> slave PB15 (CN10-26)  data
3 *          PA8 CK (CN10-23) -> slave PB13 (CN10-30)  clock
4 *          GND (CN10-20) -> slave GND (CN10-20)
5 * CR2: CLKEN=1 CPOL=0 CPHA=1 LBCL=1 (write CR2 before TE)          */
6
7#include "stm32f446xx.h"
8
9static void delay_ms(uint32_t ms)
10{
11    SysTick->LOAD = 16000 - 1;
12    SysTick->VAL  = 0;
13    SysTick->CTRL = 5;
14    while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16)));
15    SysTick->CTRL = 0;
16}
17
18int main(void)
19{
20    uint32_t t;
21
22    RCC->AHB1ENR |= (1 << 0) | (1 << 2);    /* GPIOA, GPIOC          */
23    RCC->APB2ENR |= (1 << 4);                /* USART1                */
24
25    GPIOC->PUPDR |= (1 << 26);               /* PC13 pull-up (B1)     */
26    GPIOA->MODER |= (1 << 10);               /* PA5 output (LD2)      */
27    GPIOA->MODER |= (2 << 16) | (2 << 18);    /* PA8, PA9 alternate    */
28    GPIOA->AFR[1] |= (7 << 0) | (7 << 4);     /* AF7 = CK and TX       */
29
30    USART1->CR2 = (1 << 11) | (1 << 9) | (1 << 8); /* CLKEN,CPHA,LBCL      */
31    USART1->BRR = 1667;                        /* CK = 9.6 kHz          */
32    USART1->CR1 = (1 << 3) | (1 << 13);        /* TE, UE                */
33
34    while (1) {
35        if (GPIOC->IDR & (1 << 13)) continue;
36
37        t = 0;
38        while (!(GPIOC->IDR & (1 << 13))) { delay_ms(10); t += 10; }
39
40        while (!(USART1->SR & (1 << 7)));      /* TXE                    */
41        USART1->DR = (t >= 1000) ? 'L' : 'S';
42
43        GPIOA->ODR |= (1 << 5);
44        delay_ms(100);
45        GPIOA->ODR &= ~(1 << 5);
46    }
47}
```

8 SLAVE source code `usart_sync_slave.c`

Type this over the whole contents of `Core/Src/main.c` in the slave project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

`usart_sync_slave.c`

```
1/* EXP 2 USART SYNCHRONOUS --- SLAVE --- NUCLEO-F446RE
2 * USART_CK is OUTPUT ONLY on STM32, so a USART cannot be the synchronous
3 * slave. SPI2 in slave mode is used instead: it is a shift register that
4 * runs on the clock the master supplies.
5 * Wiring : PB15 MOSI (CN10-26) <- master PA9 TX (CN10-21)
6 *          PB13 SCK (CN10-30) <- master PA8 CK (CN10-23)
7 *          GND (CN10-20) <- master GND (CN10-20)
8 * CR1: CPHA=1 CPOL=0 LSBFIRST=1 (a USART sends the LSB first) SSM=1 */
9
10#include "stm32f446xx.h"
11
12static void delay_ms(uint32_t ms)
13{
14    SysTick->LOAD = 16000 - 1;
15    SysTick->VAL = 0;
16    SysTick->CTRL = 5;
17    while (ms-- > 0) while (!(SysTick->CTRL & (1 << 16)));
18    SysTick->CTRL = 0;
19}
20
21static void blink(uint32_t d)
22{
23    for (int i = 0; i < 3; i++) {
24        GPIOA->ODR |= (1 << 5); delay_ms(d);
25        GPIOA->ODR &= ~(1 << 5); delay_ms(d);
26    }
27}
28
29int main(void)
30{
31    RCC->AHB1ENR |= (1 << 0) | (1 << 1); /* GPIOA, GPIOB */
32    RCC->APB1ENR |= (1 << 14); /* SPI2 */
33
34    GPIOA->MODER |= (1 << 10); /* PA5 output (LD2) */
35    GPIOB->MODER |= (2 << 26) | (2 << 30); /* PB13, PB15 alternate */
36    GPIOB->AFR[1] |= (5 << 20) | (5 << 28); /* AF5 = SPI2 */
37
38    SPI2->CR1 = (1 << 0) | (1 << 7) | (1 << 9) | (1 << 6);
39    /* CPHA LSBFIRST SSM(SS1=0) SPE -> slave, mode 1 */
40
41    while (1)
42        if (SPI2->SR & (1 << 0)) /* RXNE */
43            blink((uint8_t)SPI2->DR == 'L' ? 1000 : 150);
44}
```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** Open the *Board Selector* tab, type `NUCLEO-F446RE`, select it and press *Next*.
2. Name the project, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and set **Targeted Project Type = Empty**. This is the step that matters: an *Empty* project gives you the startup file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is nothing to click in CubeMX because there is no CubeMX.
3. If a dialog offers to initialise all peripherals, answer **No** – with an *Empty* project it normally does not appear at all.
4. Open `Core/Src/main.c`, select everything, delete it, and type the listing from this manual. `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the wizard already placed on the include path. There is no second file to create.
5. Build with **Ctrl + B**, connect the board, then *Run → Run As → STM32 C/C++ Application*.
6. Repeat for the second project with the other listing and flash the second board. Mark the boards **MASTER** and **SLAVE** – once they are off the PC they look identical.

If the build complains

`undefined reference to SystemInit` means the project was created without `system_stm32f4xx.c`; add `void SystemInit(void) { }` at the top of `main.c` – but only in that case, otherwise you get a duplicate symbol. `unknown type name 'uint32_t'` means the device header was not found: check that `STM32F446xx` is listed under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.

10 Running the pair with no PC attached

Nothing in these programs needs the debugger. As soon as the board is powered, the reset vector runs `main()` from flash, so the pair works on its own.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered hub. Leave jumper **JP5 on U5V**, its factory position.
3. LD3 (red) lights on each board. Keep the protocol wires and the common ground exactly as they were during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) on both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper becomes redundant – leaving it in place does no harm.

Safety

Never drive 5 V into the `+5V` pin while JP5 is on U5V *and* a USB cable is plugged in – that connects two supplies together. Pick one scheme and stay with it. BOOT0 must stay low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them as in the connection table. Check the ground wire twice – a missing ground is the single most common cause of a dead link.
3. Power both boards. LD3 (red) must be lit on each.
4. **Test 1 - short click.** Press and release B1 on the master in well under a second. The master's LD2 gives one short flash and the slave answers with three fast blinks.
5. **Test 2 - long press.** Hold B1 for at least one second, then release. The slave now blinks three times with one second on and one second off – two seconds from the start of one blink to the start of the next.
6. Time the slow pattern with a stopwatch. Repeat each test three times and record it.
7. **Test 3 - boundary.** Try a press of about one second and confirm which pattern you get. The threshold is the `t >= 1000` comparison in the master.
8. If a scope or logic analyser is available, capture the line(s) during one press and compare with the timing figure in section 6.

Trial	Press duration (ms)	Byte expected	Master LD2 flashed	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
Every received byte is bit-reversed	LSBFIRST (bit 7) is missing from the slave's <code>SPI2->CR1</code> .
Nothing is received although a scope shows a clean clock	CK and data are swapped. PA8 must go to PB13 and PA9 to PB15, not the other way round.
The clock pin stays flat	<code>CR2</code> was written after <code>CR1</code> enabled the transmitter. CLKEN, CPOL, CPHA and LBCL are only accepted while <code>TE = 0</code> .
The slave reacts once and then never again	The shift register lost its byte boundary. Press B2 RESET on the slave; if it recurs, shorten the clock wire and keep it away from the data wire.
The slave never reacts	Ground is not common. Join GND (CN10-20) of the two boards – this is by far the most frequent fault.
The master's LD2 does not flash either	The master board is not running, or B1 was not detected. Check LD3 (red) is lit and that you typed <code>GPIOC->PUPDR = (1 << 26);</code> – without the pull-up, PC13 floats and never reads as pressed.
Blink timing is wrong (much faster or slower than 2 s)	SYSCLK is not 16 MHz. Make sure no <code>SystemClock_Config()</code> or PLL setup was left in <code>main.c</code> , and that <code>SysTick->LOAD</code> is <code>16000 - 1</code> .
Both boards blink but the roles are swapped	The master listing was flashed to the slave board. Label the boards.
A long press is read as a short one	You released before 1000 ms. The threshold is the literal <code>1000</code> in the master's <code>t >= 1000</code> test – change it there if your lab asks for a different value.

13 Review questions

1. Why can an STM32 USART be a synchronous master but not a synchronous slave?
2. What do CPOL and CPHA control, and which combination is used here?
3. What is the purpose of the LBCL bit?
4. Which SPI bit makes the shift register take the LSB first, and why is it needed?
5. How many clock pulses appear on CK for one 8-bit character, and why not ten?
6. How does synchronous mode remove the baud-rate tolerance problem?
7. What do SSM and SSI do, and why is no NSS wire needed?
8. Name two real devices that are driven with USART synchronous mode.